Proposal of an automated Early Warning Score system in a New Zealand hospital

Zane Grose

06/06/2026

Media Design School

Smart Industry Applications | SIA322

Case study

Healthcare systems are becoming increasingly digitalized with electronic health records, remote patient monitoring devices, telemedicine, and automation. With the modernization through digitization of healthcare systems and services, paper-based recording systems increase the risk of transcribing errors, delay information retrieval, and are a burden to maintain. Wellington's Kenepuru community hospital continues to utilize a paper-based advisory vital signs (Early Warning Score) system which has many negatives such as transcribing errors, patient record retrieval time, and risk of missing patient deterioration between recording intervals. Automation of an Early Warning Score (EWS) system is projected to bring many benefits such as:

- Reduction in time retrieving patient history during medical emergency team activation (MET)
- Reduction of in-patient time
- Reduction of time between vital sign measurements
- Reducing risk of manual transcribing errors
- Faster alerts and notifications
- Reduction of nurse workload through offloading repetitive tasks to an automated system
- Increase of positive patient outcomes

Many of these beneficial outcomes have proven successful through previous research. One research article evaluates the "accuracy of nurse-documented vital signs and aggregated New Zealand EWS", concluding that automation of EWS brought correct EWS calculation in three quarters of all NZEWS audited (Cook et al., 2025). Another investigation performed in the US, Europe, and Australia concludes that "deploying automated advisory vital signs was associated with improvements in rapid-response team calls triggered by respiratory criteria, increased survival of patients receiving rapid-response team calls, and decreased time required for vital sign measurement and recording" (Bellomo et al., 2012).

For this project a scrum framework will be employed throughout the project, allowing the development team to meet with stakeholders and each other regularly, obtaining constant feedback, providing iterative testing and development, ultimately allowing the system to be developed for efficient use with current workflows. There are four projected sprints using the scrum framework, consisting of data collection, EWS calculation, dashboard, and alerting/notifications. These sprints will allow the development team to incrementally focus on individual parts of the system while collaborating with stakeholders to evaluate the efficiency of the developed system component. The four projected sprints are as follows:

1. Data collection
    - Vital sign monitor configuration
    - Networking (cabling, VLAN, POST/GET)
    - Edge server setup
    - Data parsing

    - API endpoints
2. Calculation, logic, and data storage
    - Application server setup
    - Network time protocol
    - Database setup
    - Data storage code
    - Synchronization service (cloud setup)
3. Dashboard
    - Dashboard engine
    - Database connection (local & cloud)
    - Edge connection to dashboard
4. Alerting & notifications
    - Alert service
    - Dashboard interface

The system must comply with the Privacy Act 2020 (Ministry of Justice, 2026) and the Health Information Privacy Code 2020 (Privacy Commissioner, 2020). This will be supported by utilizing the Health Information Standards Organisation (HISO) standards 10064:2017 (Health Information Standards Organisation, 2017) and 10029:2022 (Te Whatu Ora - Health New Zealand, 2022). These standards provide health governance guidance and a framework for health information. HL7 FHIR standard will be used to produce understandable, discoverable, and available exchange of healthcare data and information (FHIR, 2023). This will ensure New Zealand laws and healthcare standards are followed while the system ensures confidentiality, integrity, availability, and privacy concerns are appropriately met.

The system was designed with multiple functional and non-functional requirements in mind, which can be seen in tables 1 and 2. The purpose of the system is to retrieve patient data in real time, calculate in-patient EWS, store in-patient data, update dashboard views, send alerts/notifications, and update a long-term database in the cloud with patient vital information.

Table 1
System functional requirements high-level overview

FR#	Functional requirement	Description
FR1	Must calculate EWS based on vital sign data	Based on the Kenepuru Adult Vital Sign chart (Wellington ICU, 2026)
FR2	Must be able to receive communication from monitoring device	From HL7 standard communication
FR3	Must be able to connect ethernet cables to network switch	Use Cat6 cables to connect monitoring devices to switch
FR4	Must temporarily store patient data in local cache	Updates most recent patient reading in cache as a fallback method in case of network failure
FR5	Must synchronize cache records after network failure	Stored patient records should be transmitted to database after reconnection to network

FR6	Must send periodic patient data to dashboard	Depending on required periodic vital sign recording frequency defined in hospital policy
FR7	Must parse data into JSON format	From HL7
FR8	Must provide API endpoints	For event-driven retrieval of data
FR9	Must send alerts/notification to dashboard when thresholds are triggered	Alerts nurses of escalation changes
FR10	Must send data to cloud database	Uses internet (HTTPS) to send data to cloud database
FR11	Must store in-patient records in local database	For updating the cloud and dashboard views
FR12	Must maintain a network time protocol	For appropriate logging
FR13	Must allow POST request updates	For real-time data transmission to back-end server
FR14	Must be able to retrieve patient EWS information	

Table 2

System non-functional requirements

NFR#	**Non-functional requirement**	**Description**
NFR1	Must be able to access edge server through SSH	For maintenance tasks
NFR2	Must display in-patient information through dashboard	
NFR3	Must tokenize data before sending to cloud	To de-identify patient data outside of hospital network

Architecture Design

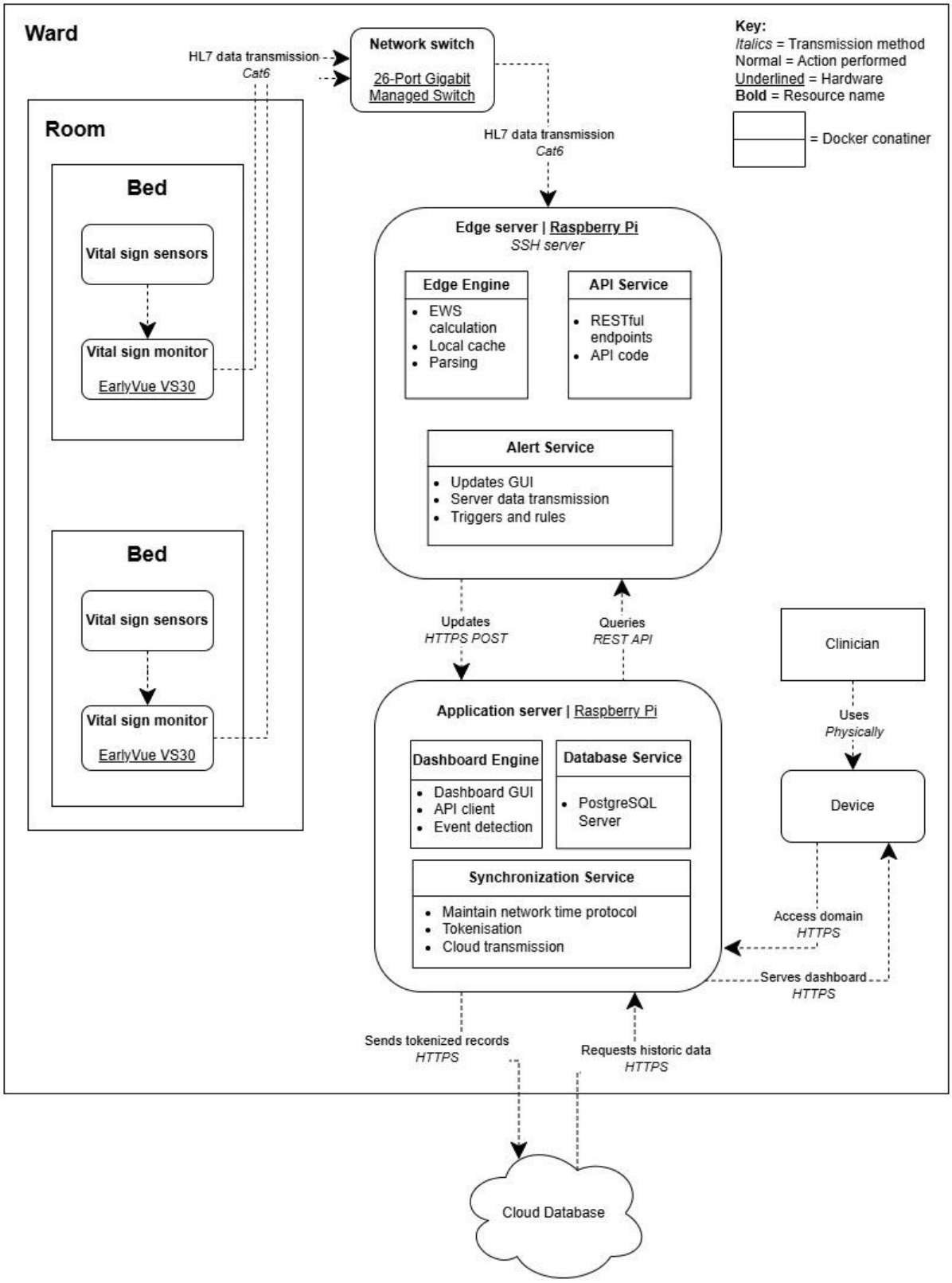


Figure. 1: High-level architecture system diagram

An edge-computing architecture was chosen for this project as it reduces network traffic, enables real-time processing of patient EWS information, provides resilience during network outages, and allows for immediate alert and notification generation. The edge server will consist of a Raspberry Pi, using Raspberry Pi OS Lite as the operating system, while running docker containers consisting of different software components (See Table 3). SSH is needed for management of the edge server, which allows remote control of the edge server for maintenance, management, and configuration.

Table 3

Containerised services to run on the Edge server

Service	Functions
Edge engine	EWS Calculations, parsing HL7 data, formatting data for API calls, local cache
API service	Supplies API endpoints, code for RESTful API
Alert service	Updates dashboard views, triggers and rules, database data transmission

The edge engine collects data, parses it, and stores it within a local cache. The application server gets information from the edge server in two ways, through POST requests and RESTful API GET requests. The alert service checks patient vital information against established rules – based on the Kenepuru Adult Vital Signs chart (Wellington ICU, 2026). If the patient's EWS score has not changed, the database will be updated, otherwise if a patient's EWS score has changed, it will send an alert or notification to the dashboard and update the database based on the change. The API service provides endpoints on the edge server, allowing the dashboard engine to utilize API endpoints to retrieve data when events are triggered. POST requests from the alert service will periodically create new patient records on the application server's database service to update the dashboard, where the time-frame is determined by a clinician on the dashboard as per hospital policies.

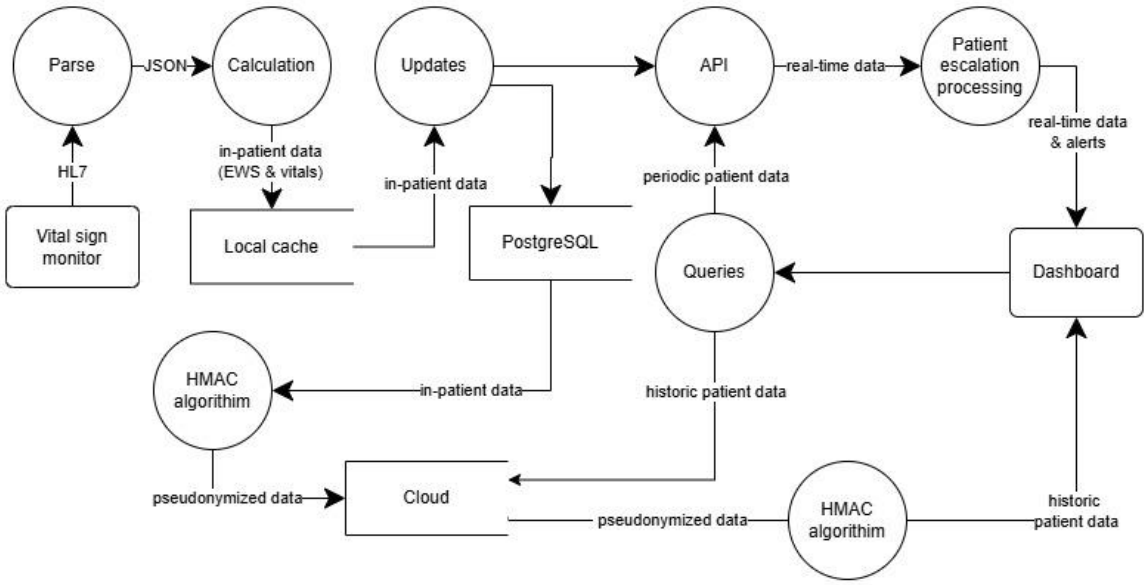


Figure. 2: System data flow diagram

Table 4

Containerized services to run on the Application server

Service	Functions
Dashboard engine	Serves dashboard information utilizing API calls, detects dashboard events
Database service	Host a local PostgreSQL database server
Synchronization service	Maintains NTP, tokenizes data, transmits data to cloud

The dashboard will be created using HTML, CSS, JavaScript, and the 'pywebview' (Sirokov, 2026) and 'requests' libraries, and Flask framework (Pallets, 2026). The application server's dashboard engine will utilize the Python programming language and the 'requests' Python library (Reitz, 2026). This library provides functionality to send or receive HTTP requests between system components. Where the API endpoints on the edge server will be provided by the Flask framework.

To ensure privacy when transferring patient information to the cloud, the synchronization service will use a HMAC-SHA256 cryptographic algorithm to generate pseudonymous identifiers from the patient's National Health Index (NHI) number. The application server will contain a secret key that is used with the patient's NHI to hash the patient's records, allowing retrieval of historical records from the cloud database by hashing a patient's NHI with the secret key and finding the associated that record in the cloud. This provides scalability through ability to integrate predictive deterioration using AI.

Hardware & Software

Several hardware and software to be procured to work, listed within Tables 5 and 6. Sensors will be connected to the vital sign monitor that transmits patient information to the edge server in real-time. Each patient's monitoring device will be connected to the network switch utilizing Cat6 cables. We avoided wireless connection due to criticality and reliability, where wireless connections can be heavily hindered by radio frequencies, physical barriers (walls), and distance. The network switch will send the ward's patient's information to the edge server; the switch is needed as it manages traffic from all monitoring devices and the edge server only has one Cat6 port.

We chose the EarlyVue VS30 vital sign monitor from Phillips due to its explicit ability to export data from the monitor and integrate it to existing systems. Though the VS30 can automatically calculate EWS, it was pertinent to create views and logic that fits Kenepuru community hospital's existing EWS system.

Docker containers provide isolation between software, allowing lightweight (compared to virtual machines) and independent deployment, maintenance, and updating of multiple software running on an individual machine, which assists with reliability by reducing the risk of software failures affecting the entire system. Using a separate VLAN (Fig. 3) in the existing network architecture supports segmentation, allowing easier network management and enhanced network security.

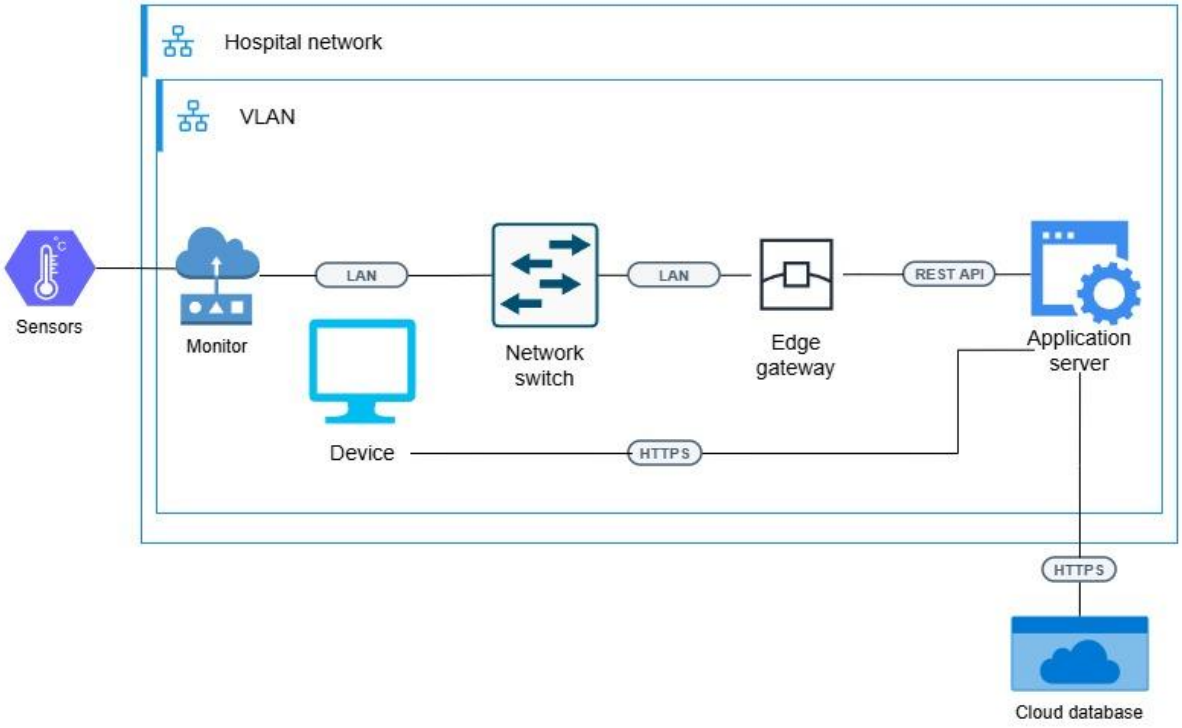


Figure. 3: System network diagram

Table 5

Required hardware

Hardware	Description	Amount required	Unit price	Total price	Reference
Vital sign monitor	Monitor that connects to sensors (EarlyVue VS30)	24	$6,424.04	$154,156	(Phillips, 2025)
Raspberry Pi 5 (16GB)	To run edge and application server software on	2	$560	$1,120	(PBTech, 2026a)
Network switch (24 port)	To connect monitors to the edge server	1	$1,194	$1,194	(PBTech, 2026b)
NVME M.2 SSD (128GB)	For the server storage	1	$100	$100	(PBTech, 2026c)
Raspberry Pi M.2 HAT	To allow connection of the SSD to the server	1	$25	$25	(PBTech, 2026d)
MicroSD (32GB)	For the edge storage	1	$25	$25	(PBTech, 2026e)
Cat6 Cable (350m)	Cable for connecting monitoring	1	$148	$148	(PBTech, 2026f)

	devices, switch, and edge server				
Cat6 plug (100 piece)	For attaching to cat6 cable	1	$34.50	$34.50	(PBTech, 2026g)

The total cost of hardware required for this project is $156,822.50. This is an enormous amount of money to implement a system. We propose instead that a trial period is performed with minimal requirements such as 4 vital sign monitors (1 large room) and other required hardware, which would instead amount to $14,374.50. This will allow for a cost-benefit analysis over a trial period to evaluate financial benefits before full-scale deployment. It is also worth noting that if existing vital sign monitors are capable of individual vital sign reading export, the cost of vital sign monitors can be avoided completely. We did not receive a quote from Phillips of a single unit of the EarlyVue VS30, we instead have researched a comparable hospital-grade vital sign monitor (Capes Medical, 2026) and used the price of that unit a guideline. This estimate is solely used for budgeting purposes and cost may differ upon procurement of the devices. This system could be restricted to high-risk patients (surgery recovery, emergency patients) in a smaller number of rooms, instead of a full-scale ward deployment. This must be discussed with stakeholders in the setup phase to evaluate the appropriate scale of deployment.

Many of the software required for this system is open-source, meaning that the code is fully available to the public and free, which helps in reducing the overall cost of the system. The only budgeting concern regarding to software is the cloud database, where we will utilize Amazon RDS PostgreSQL service (Amazon, 2026) within multiple availability zones for redundancy. The reason behind choosing Amazon is due to the dynamic pricing options they provide. We obtained data type information from the PostgreSQL documentation (PostgreSQL, 2026), listed the values needed to complete the proposed system functions, and estimated monthly cloud database costs based on those byte values. Monthly, we estimate 571,392 bytes (less than 0.001 GB) of data would be updated to the cloud database per month (Fig. 4), this means at $0.29 GB-month, a year worth of storage would amount to an estimated $0.0129. It is worth noting that this is the storage cost of data, not including database procurement, management, and hardware costs.

Value	Data Type	Size (bytes)		Value	Size (bytes)	Size (gigabytes)
tokenized_id	string	32		Patient record	128	0.000000128
date_time	datetime	12		24 patient records	3072	0.000003072
respiratory_rate	int	2		daily records (4 hourly updates p/day)	18432	0.000018432
oxygen_saturation	int	2		monthly records	571392	0.000571392
temperature	float (real)	4				
blood_pressure	int	2				
heart_rate	int	2				
alertness	int	2				
current_code	char	4		**Monthly cost**	$ 0.0002	
previous_code	char	4		**Annual cost (growing storage)**	$ 0.0129	
headers	~	24				
buffer	~	38				
Total Bytes		**128**				

Figure. 4: Price cost of storage in an Amazon RDS PostgreSQL database

Table 6

Required software

Software	Type	Description	Amount	Unit price
Cloud database	External	Long-term storage for retention and AI training, Amazon RDS for PostgreSQL using gp2 storage	1	$0.29 per GB-month
SSH server	Edge server	Allows remote connections through SSH	1	$0
SSH client	Application server	Allows connection to the edge server	1	$0
RESTful API	Edge server	Provides an endpoint for the server to retrieve data from the edge device	1	$0
Flask	Edge server	Backend framework that provides templates for creating API endpoints	1	$0
Ubuntu Server LTS	Application server	Minimalistic Linux installation built for running servers	1	$0
Raspberry Pi OS Lite	Edge server	Minimalistic text-based OS for edge server	1	$0
Docker engine	Edge & Application servers	Containerisation software for running isolated software	2	$0
Python	Edge & Application servers	Programming language to prototype and implement logic and algorithms	2	$0
Pywebview library	Application server	Provides library to display HTML contents in a native GUI	1	$0
Requests library	Application server	Provides library that allows easy integration of HTTP requests	2	$0
PostgreSQL Server	Application server	Provides a local database server for patient information	1	$0

There are many system types embedded into this solution consisting of time-driven, event-driven, monitoring, reactive, decision-support, and human-centred systems. Through periodic data transfer between the edge and application servers and updates to the cloud, the system utilizes a time-dependent system to update records. The proposed system also displays dashboard alerts and notification events, supporting event-driven system design. Dashboard views support human-centred and monitoring system designs through alerts, notifications, and mandatory clinical action views. Through triggers, rules, and actions within the alert service,

this design supports the reactive-based system design described above and event-based design through API calls from dashboard graphical interfaces.

Stakeholders

The scrum master oversees the steps in the system development process, requirement criteria, software and hardware procurement, and team management including coaching team members in self-management and cross-functionality. This is a critical role as they are the overseer of the process, development team, and communication with the client.

The development team is a team of software engineers that create, manage, implement, and maintain the system that is deployed. This team is critical as they need to ensure self-management, flexibility, and deliverability throughout the system lifecycle.

The hospital representative is the stakeholder that makes high-level decisions on behalf of the hospital. This is a critical stakeholder as data retention, security, budgeting, laws, and policies will be brought to this person. They act as a liaison between tertiary stakeholders such as auditors, lawmakers, regulators, and the hospital.

Nurses and doctors are the end-users that will utilize the system to ensure a smoother workflow of their daily jobs. It is critical that a select few nurses and doctors with responsibility are kept informed about the project so throughout the process and after it is deployed, the system quantify brings benefits to their workflow.

IT personnel refer to the hospital IT personnel and is imperative to include within the development and maintenance process. These are the personnel that will help in creation of VLANS and connections between components.

The entire system runs on patient data and though patients will not be a part of the development process, it is critical to ensure policies, data lifecycles, and agreement documents are created so patients can understand how their data is used, managed, processed, stored, and destroyed within the system.

Rules, triggers, and actions

Escalation code is a tracked throughout the entire lifecycle of the vital sign data throughout its life while it's inside the edge engine. This escalation code directly correlates with mandatory actions, advisory messages, and alerts/notifications that allow clinicians to make clinical decisions about a patient. If a patient has a total EWS score of 1, then the escalation code is in ORANGE. Conversely, if the total EWS score is 3, but that 3 comes from respiratory rate being at 30, then the escalation code changes to 'pink'. Within the Kenepuru community hospital adult vital signs chart, any value that appears in the 'blue' zone automatically enables a patient into MET activation.

Table 7

Example rules, trigger, and actions for vital sign values, score is the vital sign's EWS score. For an extensive list of rules see (Wellington ICU, 2026).

Vital sign name	Rule	Trigger	Action

Respiratory rate (breaths per minute)	>= 9 and <= 11	*Value* is between or equal to 9 and 11	score = score + 1
Oxygen saturation (percentage)	Value == 94 or value == 95	Value is either 94 or 95	score = score + 1
Temperature (Celsius)	Value >= 36.00 and value <= 37.99	Value is between or equal to 36 and 37.99	score = score + 0
Blood pressure (systolic value)	Value >= 220	Value is greater than or equal to 220	score = score + 3
Heart rate (beats per minute)	Value >= 30 and value <=39 OR Value < 30	Value is between or equal to 30 and 39 OR Value is lower than 30	score = score + 10
Alertness (APVU)	Value = 3	Value is equal to 3	score = score + 3
Escalation code (colour)	Value >= 10	Value is greater or equal to 10	code = blue
Mandatory action (advised actions)	Value >= 10 OR Code = Blue	Value is greater or equal to 10	Send alert: - Dial 777 - State 'Medical Emergency' and your location - Support airway, breathing & circulation - Consider transfer to Wellington Regional Hospital
Escalation code	Current code > previous code	The current code is greater than the previous code	status = current code
Alertness	Value >= 2	Value is greater or equal to 2	score = score + 2

Project Timeline

The setup phase of the project will start by performing an initial meeting with stakeholders, explaining each sprint's goals, hardware, software, networking, security, privacy, and regulation requirements. For development team members, this phase will define sprint goals, requirements, and duties for individual system components within each sprint. The SCRUM backlog, which consists of SCRUM artifacts, will be established and help in identifying what needs to be completed to meet goal criteria. Daily SCRUM meeting times and sprint retrospectives will be established which promotes iterative collaboration between members, allowing for dynamic development of components.

Maintenance will be performed throughout the lifecycle of the system which includes management of data within the databases (local and cloud), updating system components

(libraries, software), regular data privacy and security updates, and audits of the system to ensure originally set goals are being consistently met. The system should constantly be evaluated for accuracy, reliability, and responsiveness of system functionality. Policies and agreements between the hospital and patients should be re-evaluated every half year to ensure changes within the system relating to patient data are being represented to patients, nurses, and doctors.

During the end-of-life of the system, data within the database must be retained for 10 years from the last encounter with the patient, unless transferred to another doctor or to the patient, as per HIPC (Privacy Commissioner, 2026). This data can be stored within the cloud with HMAC identifiers, which means that the key used to hash patient NHI's should be retained securely for access to cloud-based records. After this 10-year period, all data should be disposed of permanently, including any backups of the data. Stakeholders should be notified of end-of-life before it occurs with documentation on how, when, why, and what to provide quantitative guidance on the end-of-life process for users and the development team.

The timeline for each sprint in the project's development varies. This is highly dependent on the components being developed within the sprint. For instance, sprint 2 will take longer as core components are developed, whereas sprint 1 is shorter as it is primarily configuration tasks. Each sprints estimated timelines are represented in Figure 5, where 'duration' is days. The bar chart was not included due to visibility issues in the document.

At the end of each sprint, a sprint review is conducted with the scrum master, development team, and stakeholders to evaluate the status of the developed component. Though there is a rough outline of the project timeline, SCRUM is an empirical process that focuses on transparency, inspection, and adaptation. This allows for iterative requirement gathering, system modifications, and stakeholder feedback throughout the development process.

Name	Begin date	End date	Duration
Sprint 1 - Data collection	1/06/26	20/07/26	36
Vital sign monitor config	1/06/26	9/06/26	7
Networking	8/06/26	16/06/26	7
Edge gateway setup	17/06/26	25/06/26	7
Data parsing	26/06/26	2/07/26	5
API endpoints	3/07/26	20/07/26	12
Sprint 2 - Calculation, logic, and data storage	21/07/26	23/09/26	47
Application server setup	21/07/26	3/08/26	10
Network time protocol	4/08/26	7/08/26	4
PostgreSQL Database	10/08/26	20/08/26	9
Data storage code	21/08/26	3/09/26	10
Synchronization service	4/09/26	23/09/26	14
Sprint 3 - Dashboard	24/09/26	19/11/26	41
Dashboard engine	24/09/26	22/10/26	21
Database connection	23/10/26	5/11/26	10
Edge connection to dashboard	6/11/26	19/11/26	10
Sprint 4 - Alerting & notifications	20/11/26	29/12/26	28
Alert service	20/11/26	9/12/26	14
Dashboard interface	10/12/26	29/12/26	14

Figure. 5: Timeline of sprint phases

References

Amazon. (2026, 06). *Amazon RDS for PostgreSQL pricing*. Retrieved from https://aws.amazon.com/rds/postgresql/pricing/?pg=pr&loc=3

Bellomo, R., Ackerman, M., Bailey, M., Beale, R., Clancy, R., Clancy, G., . . . Tangkau, P. (2012). A controlled trial of electronic automated advisory vital signs monitoring in general hospital wards. *Critical Care Medicine, 40*(8), 2349-2361. https://doi.org/10.1097/CCM.0b013e318255d9a0

Capes Medical. (2026, 06). *Biolight S10 Paitent Monitor with ECG, HR, RESP, NIBP, SPO2 and Temperature*. Retrieved from https://www.capesmedical.co.nz/shop/Patient+Monitoring/Multi+Parameter+Monitors/Biolight+S10+Paitent++Monitor+with+ECG+HR+RESP+NIBP+SPO2+and+Temperature.html

Cook, J. K., Biggs, R., Eathorne, A., Kingi, T., Shortt, N., Kivi, N., . . . Kirton, L. (2025). A single-centre audit of early warning scores and medical emergency team activation in medical and surgical admissions at Wellington Regional Hospital. *Postgraduate Medical Journal, 102*(1203), 95-102. https://doi.org/10.1093/postmj/qgaf106

FHIR. (2023, 03 26). *Documentation Index.* Retrieved from HL7: https://www.hl7.org/fhir/documentation.html

Health Information Standards Organisation. (2017, 08). *HISO 10064:2017 Health Information Governance Guidelines.* Ministry of Health. Retrieved from https://static.info.content.health.nz/docs/HISO/HISO%2010064%20Health%20Information%20Governance%20Guidelines.pdf

Ministry of Justice. (2026, 05 01). *Privacy Act 2020*. Retrieved from New Zealand Legislation: https://www.legislation.govt.nz/act/public/2020/31/en/latest/#LMS23223

Pallets. (2026). *Welcome to Flask - Flask Documentation (3.1.x)*. Retrieved from Palletsprojects: https://flask.palletsprojects.com/en/stable/

PBTech. (2026a). *Raspberry Pi 5 16GB LPDDR4 2.4GHz Quad-Core ARM Cortex-A76 - Dual 4K Displays - LPDDR4X-4267 SDRAM - PCIe 2.0 - Dual-band 802.11ac Wi-Fi - Bluetooth - Support PoE+*. Retrieved 05 31, 2026, from https://www.pbtech.com/pacific/product/SEVRBP0505/Raspberry-Pi-5-16GB-LPDDR4-24GHz-Quad-Core-ARM-Cor

PBTech. (2026b, 05). *MikroTik CRS326-24S+2Q+RM Cloud Router 26 Port 10 Gbps Fibre Switch with 40 Gbps Uplink*. Retrieved from https://www.pbtech.co.nz/product/SWHMKT690046/MikroTik-CRS326-24S2QRM-Cloud-Router-26-Port-10-Gb

PBTech. (2026c, 05 31). *128GB M.2 NVMe Internal 2242 SSD*. Retrieved from https://www.pbtech.co.nz/product/HDDOEM0124/128GB-M2-NVMe-Internal-2242-SSD---with-single-notc

PBTech. (2026d, 05 31). *Raspberry Pi 5 Official M.2 HAT+ M Key Supports 2230 or 2242 Form Factor, M.2 - Format PCIe / NVMe SSD and Other PCIe Accessories*. Retrieved from https://www.pbtech.co.nz/product/SEVRBP0533/Raspberry-Pi-5-Official-M2-HAT-M-Key-Supports-2230?src=raspberrypi

PBTech. (2026e, 05 31). *ADATA Premier microSDXC Memory Card - 64GB*. Retrieved from https://www.pbtech.co.nz/product/MEMADC20020/ADATA-Premier-microSDXC-Memory-Card---64GB-Include

PBTech. (2026f, 05 31). *8Ware CAT6-EXT350BLU Cat6 Ethernet Cable - 350m - Blue*. Retrieved from https://www.pbtech.co.nz/product/CAB8WR0225/8Ware-CAT6-EXT350BLU-Cat6-Ethernet-Cable---350m

PBTech. (2026g, 05 31). *Cruxtec CAT6 UTP RJ45 Plug - 100 pieces / Jar*. Retrieved from https://www.pbtech.co.nz/product/CABCXT800100/Cruxtec-CAT6-UTP-RJ45-Plug---100-pieces--Jar

Phillips. (2025). *EarlyVue VS30*. Retrieved from https://www.philips.co.nz/healthcare/product/HC863359/earlyvuevs30vitalsignsmonitor

PostgreSQL. (2026, 06). *Chapter 8. Data Types*. Retrieved from https://www.postgresql.org/docs/current/datatype.html

Privacy Commissioner. (2020). *Health Information Privacy Code 2020*.

Reitz, K. (2026, 06). *Requests.* Retrieved from Github: https://github.com/psf/requests

Sirokov, R. (2026, 06). *pywebview*. Retrieved from https://pywebview.flowrl.com/

Te Whatu Ora - Health New Zealand. (2022, 12). *HISO 10029:22 Health Information Security Framework*. Retrieved from https://static.info.content.health.nz/docs/HISO/HISO%2010029%20Health%20Information%20Security%20Framework.pdf

Wellington ICU. (2026). *Kenepuru Adult Vital Signs Chart*. Retrieved from https://wellingtonicu.com/Data/EWS2/KeneFBC.pdf